OS WEEK 6

1. **prak6-1.c**
    - Mulai
    - Buat Thread 1
    - Buat Thread 2
    - Thread 1 & 2 jalan bersamaan (saling berebut increment cnt)
    - Tunggu kedua thread selesai
    - Cek nilai cnt
    - Cetak hasil
    - Selesai

    Operasi cnt++ tidak atomik sehingga 2 thread bisa baca nilai sama. Terjadi increment ganda hanya menaikkan 1 sehingga hasil akhir < 20 juta.

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

// Mendefinisikan konstanta NITERS dengan nilai 10.000.000
// Konstanta ini akan digunakan sebagai jumlah iterasi loop di setiap thread
#define NITERS 10000000

// Deklarasi prototipe fungsi count yang akan dijalankan oleh thread
// Fungsi ini menerima argumen bertipe void* dan mengembalikan void*
void *count(void *arg);

// Variabel global cnt (penghitung) yang akan diakses dan dimodifikasi oleh kedua thread
// Karena bersifat global dan tidak dilindungi, ini akan menyebabkan race condition
unsigned int cnt = 0;

int main()
{
    // Deklarasi dua variabel untuk menampung identifier (ID) thread
    // Catatan: seharusnya tid1 dan tid2, tapi tertulis tidl (mungkin typo)
    pthread_t tidl, tid2;

    // Membuat thread pertama (tidl) yang akan menjalankan fungsi count
    // Parameter: &tidl (penyimpan ID thread), NULL (atribut default),
    //            count (fungsi yang dijalankan), NULL (argumen fungsi)
    pthread_create(&tidl, NULL, count, NULL);

    // Membuat thread kedua (tid2) yang juga menjalankan fungsi count yang sama
    pthread_create(&tid2, NULL, count, NULL);

    // Menunggu thread pertama (tidl) selesai dieksekusi
    // pthread_join akan memblokir program utama hingga thread tidl selesai
    pthread_join(tidl, NULL);

    // Menunggu thread kedua (tid2) selesai dieksekusi
    pthread_join(tid2, NULL);

    // Memeriksa apakah nilai cnt sesuai dengan yang diharapkan
    // Yang diharapkan: setiap thread melakukan increment sebanyak NITERS kali
    // Dua thread = 2 * NITERS = 20.000.000
    if (cnt != (unsigned)NITERS*2)
        // Jika tidak sesuai (karena race condition), cetak pesan kesalahan
        printf("Wrong... cnt = %d\n", cnt);
    else
        // Jika sesuai (hampir tidak pernah terjadi tanpa sinkronisasi), cetak sukses
        printf("OK...\tcnt=%d\n", cnt);
```


```c
        // Jika sesuai (hampir tidak pernah terjadi tanpa sinkronisasi), cetak sukses
        printf("OK...\tcnt=%d\n", cnt);
```


```c
    // Mengakhiri program dengan kode 0 (menandakan sukses)
    exit(0);
}

// Implementasi fungsi count yang akan dijalankan oleh thread
void *count(void *arg)
{
    // Variabel lokal sebagai counter loop
    int i;

    // Loop sebanyak NITERS kali (10 juta kali)
    for (i=0; i<NITERS; i++)

        // INI ADALAH BAGIAN KRITIS:
        // Operasi increment pada variabel global cnt
        // Operasi ini TIDAK ATOMIK (terdiri dari: baca, tambah, tulis)
        // Karena dua thread melakukannya bersamaan, bisa terjadi interleaving
        // yang menyebabkan hasil akhir tidak sesuai harapan (race condition)
        cnt++;

    // Fungsi count tidak mengembalikan nilai (return NULL secara implisit)
}
```

2. **prak6-2.c**

perbaikan dari prak6-1 menggunakan mutex. Harusnya nanti sudah ok hasilnya.

- Thread A masuk loop
- pthread_mutex_lock(&mutex)
- berhasil mengunci
- Thread A masuk critical region
- melakukan cnt++
- Thread B mencoba pthread_mutex_lock(&mutex)
- terblokir (menunggu)
- Thread A selesai cnt++
- pthread_mutex_unlock(&mutex)
- membuka kunci
- Thread B sekarang berhasil mengunci
- masuk critical region
- cnt++
- Proses berulang hingga 10 juta kali untuk masing-masing thread

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

// Mendefinisikan konstanta NITERS dengan nilai 10.000.000
#define NITERS 10000000

// Deklarasi prototipe fungsi count
void *count(void *arg);

// Variabel global cnt (penghitung) yang akan diakses oleh kedua thread
unsigned int cnt = 0;

// MUTEX: Variabel global untuk sinkronisasi
// PTHREAD_MUTEX_INITIALIZER untuk inisialisasi mutex secara statis
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main()
{
    // Deklarasi dua variabel untuk ID thread
    pthread_t tid1, tid2;  // Diperbaiki dari tidl menjadi tid1

    // Membuat thread pertama
    pthread_create(&tid1, NULL, count, NULL);

    // Membuat thread kedua
    pthread_create(&tid2, NULL, count, NULL);

    // Menunggu thread pertama selesai
    pthread_join(tid1, NULL);

    // Menunggu thread kedua selesai
    pthread_join(tid2, NULL);

    // Memeriksa apakah nilai cnt sesuai (20.000.000)
    if (cnt != (unsigned)NITERS * 2)
        printf("Wrong... cnt = %d\n", cnt);
    else
        printf("OK...\tcnt = %d\n", cnt);

    // Mengakhiri program
    exit(0);
}
```

```c
// Implementasi fungsi count dengan mutex
void *count(void *arg)
{
    int i;

    // Loop sebanyak NITERS kali
    for (i = 0; i < NITERS; i++)
    {
        // MENGUNCI MUTEX sebelum memasuki critical region
        // Hanya satu thread yang bisa melewati baris ini
        pthread_mutex_lock(&mutex);

        // ---------- CRITICAL REGION ----------
        // Operasi increment pada variabel global cnt
        // Sekarang aman karena hanya satu thread yang bisa mengakses di waktu yang sama
        cnt++;
        // ---------- AKHIR CRITICAL REGION ----------

        // MEMBUKA MUTEX setelah keluar dari critical region
        // Thread lain yang menunggu sekarang bisa masuk
        pthread_mutex_unlock(&mutex);
    }

    // Fungsi count selesai
}
```

3. **prak6-2-busy.c**
    - Thread A masuk loop
    - pthread_mutex_trylock()
    - berhasil
    - masuk critical region
    - Thread B masuk loop
    - pthread_mutex_trylock()
    - agal (EBUSY)
    - Thread B masuk ke while loop
    - terus mencoba lagi dan lagi tanpa istirahat
    - Thread A selesai cnt++
    - pthread_mutex_unlock()
    - Thread B pada percobaan berikutnya berhasil
    - masuk critical region
    - Proses berulang hingga selesai

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <errno.h>  // Untuk menggunakan EBUSY

// Mendefinisikan konstanta NITERS dengan nilai 10.000.000
#define NITERS 10000000

// Deklarasi prototipe fungsi count
void *count(void *arg);

// Variabel global cnt (penghitung) yang akan diakses oleh kedua thread
unsigned int cnt = 0;

// Mutex global untuk sinkronisasi
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main()
{
    // Deklarasi dua variabel untuk ID thread
    pthread_t tid1, tid2;

    // Membuat thread pertama
    pthread_create(&tid1, NULL, count, NULL);

    // Membuat thread kedua
    pthread_create(&tid2, NULL, count, NULL);

    // Menunggu thread pertama selesai
    pthread_join(tid1, NULL);

    // Menunggu thread kedua selesai
    pthread_join(tid2, NULL);

    // Memeriksa apakah nilai cnt sesuai (20.000.000)
    if (cnt != (unsigned)NITERS * 2)
        printf("Wrong... cnt = %d\n", cnt);
    else
        printf("OK...\tcnt = %d\n", cnt);

    // Mengakhiri program
    exit(0);
}
```

```c
// Implementasi fungsi count dengan metode busy-waiting (polling)
void *count(void *arg)
{
    int i;
    int ret;  // Variabel untuk menampung return value dari pthread_mutex_trylock()

    // Loop sebanyak NITERS kali
    for (i = 0; i < NITERS; i++)
    {
        // BUSY-WAITING / POLLING:
        // Mencoba mengunci mutex dengan trylock (non-blocking)
        // Jika mutex sedang terkunci (EBUSY), loop akan terus berulang
        // Ini membuat CPU terus bekerja selama menunggu (busy-waiting)
        while ((ret = pthread_mutex_trylock(&mutex)) == EBUSY)
        {
            // Tidak melakukan apa-apa, hanya terus memeriksa
            // CPU akan sibuk selama loop ini berjalan
        }

        // ---------- CRITICAL REGION ----------
        // Setelah berhasil mendapatkan lock, lakukan increment
        cnt++;
        // ---------- AKHIR CRITICAL REGION ----------

        // Membuka kunci mutex
        pthread_mutex_unlock(&mutex);
    }

    // Fungsi count selesai
}
```

4. **prak6-3.c**
   - Thread 1 lock mutex_1
   - Thread 2 lock mutex_2
   - Thread 1 coba lock mutex_2
   - terblokir (dipegang Thread 2)
   - Thread 2 coba lock mutex_1
   - terblokir (dipegang Thread 1)
   - DEADLOCK - kedua thread saling menunggu selamanya

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NITERS 100000000

pthread_mutex_t mutex_1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex_2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_1(void *arg);
void *thread_2(void *arg);

int main()
{
    pthread_t tid1, tid2;

    pthread_create(&tid1, NULL, thread_1, NULL);
    pthread_create(&tid2, NULL, thread_2, NULL);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    return 0;
}

void *thread_1(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++)
    {
        pthread_mutex_lock(&mutex_1);  // Lock mutex_1 dulu
        pthread_mutex_lock(&mutex_2);  // Lock mutex_2 kedua
        printf("Thread 1 \n");
        pthread_mutex_unlock(&mutex_1); // Unlock mutex_1 saja
        //Mutex2 tidak di unlock nihh. Deadlock
    }
}
```

```c
void *thread_2(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++)
    {
        pthread_mutex_lock(&mutex_2);  // Lock mutex_2 dulu
        pthread_mutex_lock(&mutex_1);  // Lock mutex_1 kedua
        printf("Thread 2 \n");
        pthread_mutex_unlock(&mutex_1); // Unlock mutex_1
        pthread_mutex_unlock(&mutex_2); // Unlock mutex_2
    }
}
```

5. **prak6-3-sem.c**
   Perbaikan prak6-3 menggunakan semaphore tanpa ada deadlock atau race condition. Konstanta NITERS nya ku kurangi jadi 100 untuk contoh

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define NITERS 100

sem_t resource_A;
sem_t resource_B;
sem_t turn_1;  // Semaphore untuk thread 1
sem_t turn_2;  // Semaphore untuk thread 2

void *thread_1(void *arg);
void *thread_2(void *arg);

int main()
{
    pthread_t tid1, tid2;

    sem_init(&resource_A, 0, 1);
    sem_init(&resource_B, 0, 1);
    sem_init(&turn_1, 0, 1);  // Thread 1 jalan dulu
    sem_init(&turn_2, 0, 0);  // Thread 2 menunggu

    pthread_create(&tid1, NULL, thread_1, NULL);
    pthread_create(&tid2, NULL, thread_2, NULL);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    sem_destroy(&resource_A);
    sem_destroy(&resource_B);
    sem_destroy(&turn_1);
    sem_destroy(&turn_2);

    return 0;
}
```

```c

void *thread_1(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++)
    {
        sem_wait(&turn_1);  // Tunggu giliran thread 1

        sem_wait(&resource_A);
        sem_wait(&resource_B);

        printf("Thread 1\n");

        sem_post(&resource_B);
        sem_post(&resource_A);

        sem_post(&turn_2);  // Beri giliran ke thread 2
    }

    return NULL;
}

void *thread_2(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++)
    {
        sem_wait(&turn_2);  // Tunggu giliran thread 2

        sem_wait(&resource_A);
        sem_wait(&resource_B);

        printf("Thread 2\n");

        sem_post(&resource_B);
        sem_post(&resource_A);

        sem_post(&turn_1);  // Beri giliran ke thread 1
    }

    return NULL;
}
```

6. **practice.c**

   ***NOTES: Kodenya belum diperbaiki. Untuk kode perbaikan silahkan implementasi solusi masing-masing. Tidak perlu buat file baru. Perbaiki aja langsung di practice.c**

   - Ada 5 philosopher duduk melingkar.
   - Setiap philosopher butuh 2 fork untuk makan.
   - Fork direpresentasikan oleh mutex agar tidak dipakai bersamaan.
   - Philosopher melakukan siklus:
     - Thinking
     - Hungry
     - Take left fork
     - Take right fok
     - Eating
     - Release forks

   KODE ASLI:

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

#define NUM_PHIL 5
#define SLEEP_TIME 2

// Array mutex yang merepresentasikan fork
// Setiap fork adalah resource yang harus dikunci sebelum digunakan
pthread_mutex_t forks[NUM_PHIL];

// Fungsi yang akan dijalankan oleh setiap thread philosopher
void* philosopher(void* num) {
    int id = *(int*)num;                // Mengambil ID philosopher dari parameter
    int left_fork = id;                 // Fork kiri adalah fork dengan indeks sama dengan ID philosopher
    int right_fork = (id + 1) % NUM_PHIL; // Fork kanan adalah fork berikutnya (melingkar)

    while (1) { // Loop terus menerus (philosopher terus berpikir dan makan)

        printf("Philosopher %d is thinking.\n", id);  // Menampilkan bahwa philosopher sedang berpikir
        sleep(rand() % SLEEP_TIME);                   // Delay acak untuk simulasi waktu berpikir

        printf("Philosopher %d is HUNGRY.\n", id);    // Philosopher menjadi lapar

        pthread_mutex_lock(&forks[left_fork]);        // Mengunci fork kiri agar tidak dipakai philosopher lain
        printf("Philosopher %d picked up left fork %d.\n", id, left_fork); // Informasi fork kiri diambil

        sleep(1);                                     // Delay kecil untuk simulasi waktu

        pthread_mutex_lock(&forks[right_fork]);       // Mengunci fork kanan
        printf("Philosopher %d picked up right fork %d and is EATING.\n", id, right_fork); // Mulai makan

        sleep(rand() % SLEEP_TIME);                   // Simulasi waktu makan

        pthread_mutex_unlock(&forks[right_fork]);     // Melepaskan fork kanan setelah selesai makan
        pthread_mutex_unlock(&forks[left_fork]);      // Melepaskan fork kiri

        printf("Philosopher %d finished eating and put down forks.\n", id); // Informasi selesai makan
    }
}

int main() {

    pthread_t phils[NUM_PHIL];   // Array untuk menyimpan thread philosopher
    int phil_ids[NUM_PHIL];      // Array untuk menyimpan ID masing-masing philosopher

    // Inisialisasi mutex untuk setiap fork
    for (int i = 0; i < NUM_PHIL; i++) {
        pthread_mutex_init(&forks[i], NULL); // Menginisialisasi mutex fork ke-i
    }

    // Membuat thread philosopher
    for (int i = 0; i < NUM_PHIL; i++) {
        phil_ids[i] = i;  // Memberikan ID ke philosopher

        // Membuat thread baru yang menjalankan fungsi philosopher
        pthread_create(&phils[i], NULL, philosopher, &phil_ids[i]);
    }

    // Menunggu semua thread selesai (join)
    for (int i = 0; i < NUM_PHIL; i++) {
        pthread_join(phils[i], NULL);  // Program utama menunggu thread philosopher selesai
    }

    return 0; // Program selesai
}
```

KODE PERBAIKAN:
SILAHKAN DIBUAT SENDIRI SESUAI SOLUSI MASING-MASING